

Machine Learning Models

Chapter 1: The Machine Learning Landscape

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Gurelien Geron, 3e, O'Reilly Media, 2022.

Models in Machine Learning

- A machine learning model is a mathematical representation or a program that learn from a dataset and can then generalize to make decisions for a previously unseen dataset.
- One can build more than one model for the given dataset, can then choose the best one or aggregate the results of all of them.
- The resulting final model should generalize well on new / real world data.

Model Parameters vs. Hyperparameters

- A **model or training parameter** is a configuration variable that is **internal to the model** and whose value can be estimated from data.
 - They are estimated or learned from data.
 - They are often not set manually by the practitioner; only initial values may be set by the practitioner.
 - They are required by the model when making predictions.
 - A machine learning model with a set number of model or training parameters is called a **parametric model**; one without a set number of model or training parameters is referred to as **non-parametric**.
- A **hyperparameter** is a configuration that is **external to the model** and whose value cannot be estimated from data.
 - These are the setting which are used to control the behavior of a learning algorithm.
 - The values of hyperparameters are not adapted by the learning algorithm itself.
 - They are often specified by the practitioner, through heuristics and hand tuning.
 - We cannot know the best value for a model hyperparameter on a given problem. We may use rules of thumb, copy values used on other problems, or search for the best value by trial and error.

Model Generalization

- The **ability to perform well on previously unobserved inputs** is called **generalization**.
- The only way to know how well a model will generalize to new cases is to actually try it out on new cases.
- A machine learning model should perform well not only on the training data but also on new, unseen data, this ability is called generalization.
- **To estimate generalization performance during development, the dataset is split into training and test (or validation) sets (commonly 80/20, but this depends on dataset size).**
- **Train Error** is computed on data the model learned from.
- **Test/Generalization Error** indicates expected performance on new inputs and tells how well the model generalizes.
- Models are compared based on their test/generalization performance, not just training accuracy.

Underfitting vs Overfitting

- **Underfitting:** High error even on training data (model too simple or under-trained).
- **Overfitting:** Very low training error but significantly higher test error (model memorizes training data instead of learning patterns – over trained).

Quick Interpretation Guide

Scenario	Diagnosis
Train and test errors both high and close	Underfitting
Train error low, test error high with large gap	Overfitting
Train error low, test error slightly higher but stable	Good generalization

- A good model should:
 - Achieve **low training error**.
 - Keep the **train-test error gap small**.

How much train error is acceptable?

- Not all tasks can achieve 0% error. To judge whether performance is “good”, compare with human-level accuracy, used as a proxy for Bayes error (theoretical minimum error).

How Much Train–Test Gap is Acceptable?

- There is no fixed universal threshold, but acceptability depends on various factors.
- If both train and test errors are low, a small gap is acceptable (e.g., 2% vs 4%).
- As a rule of thumb, if test error increases more than ~20–30% *relative* to train error, overfitting is likely.
- Small datasets naturally have a larger gap. If adding more data or regularization reduces the gap, it confirms overfitting.

Solutions

- Overfitting → Apply regularization, dropout, collect more data, early stopping.
- Underfitting → Increase model capacity, train longer, use better features, change the model.

No Free Lunch (NFL) Theorem

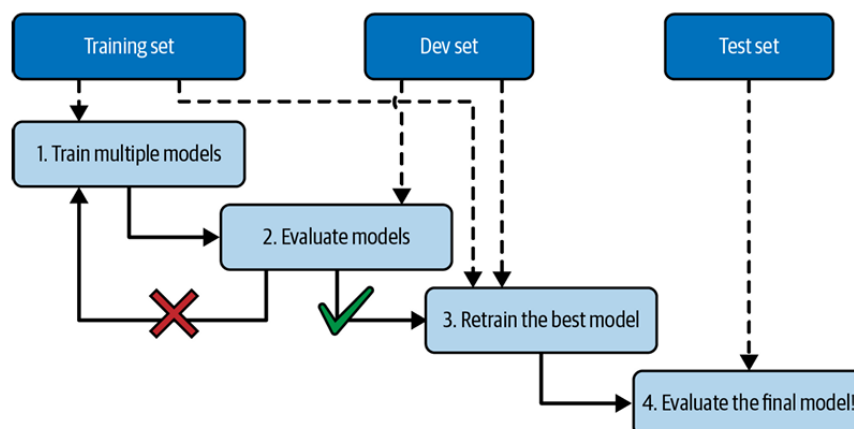
- It emphasizes that no single model is universally best for all problems [David Wolpert, 1996].
- “No Free Lunch” because there is no model that gives good performance “for free” on all problems.
- Every model is a simplified view of reality and relies on assumptions about the data; an assumption that works well in one domain may perform poorly in another.
- Therefore, there is no universally optimal model, linear models may suit some datasets, while neural networks may suit others.
- The practical way to choose a model is through domain knowledge and model selection techniques such as cross-validation. This is why having a diverse set of models and algorithms is essential in machine learning.

Validation Techniques

- Commonly Used Validation Techniques
 - Hold-out Validation
 - K-Fold Cross Validation

Hold-out Validation

- Simply hold out a validation set from the training set to evaluate several candidate models and select the best one.
- The dataset is divided into three subsets: the train set, the validation set, and the test set.
- The training set is used to train the algorithm and finds best values for the model parameters.
- The validation set or developer/dev set is used to guide the selection of hyperparameters. Train multiple models with various hyperparameters on the new reduced training set and you select the model that performs best on the validation set.
- Both train and validation sets aim to minimize the train error.
- After all hyperparameter optimization is complete, train the best model on the full training set (including the validation set), and this gives you the final model; the generalization error is now estimated using the test set.
- Once the model has been finalized, it is train on complete dataset before the final launch.
- This is the simplest evaluation method and is widely used in machine learning projects.



Issues

- Dividing the dataset into a three fixed sets can be problematic if the dataset is small because:
 - A small training set may lead to underfitting. Also it is not ideal to compare candidate models trained on a much smaller training set.
 - If validation or/and test sets are small, the model evaluations will be imprecise: you may end up selecting a suboptimal model by mistake.
 - Test error rates could be highly variable, totally depending on which observations end up in the training set and test set.
- One way to solve this problem is to perform repeated cross-validation.
- Use all of the examples in the estimation of the mean test error, at the price of increased computational cost.
- These procedures are based on the idea of repeating the training and testing computation on different randomly chosen subsets or splits of the original dataset.
- The most common of these is the k-fold cross-validation procedure.

k-Fold Cross-Validation

- In k-fold cross-validation, the available dataset is divided into k approximately equal-sized folds.
- In each iteration, one fold is used as the validation set, while the remaining k-1 folds are used for training.
 - In the first iteration, the first fold is selected as the validation set and the model is trained on the remaining k-1 folds.
 - The validation error is then computed.
 - In the second iteration, the second fold is used as the validation set, and the remaining k-1 folds are used for training.
 - This process continues until each fold has been used once as the validation set.
- The final cross-validation error is estimated by averaging the validation errors across all k iterations:

$$CV_k = \frac{1}{k} \sum_{i=1}^k MSE_i$$



- Overfitting can be assessed by comparing the average training error and the average validation error obtained through cross-validation; a large gap between them indicates poor generalization.
- Each data point is used exactly once for validation and k-1 times for training, making efficient use of the data.
- Common choices such as k = 5 or k = 10 provide a good balance between bias and variance and are widely used in practice.
- The main disadvantage of k-fold cross-validation is that the model must be trained k times, making it computationally more expensive than the hold-out method.
- Standard k-fold cross-validation also does not guarantee equal class proportions in each fold. This issue is addressed by Stratified K-Fold Cross-Validation, which preserves class distributions across folds.

Stratified k-Fold Cross-Validation

- A variation of k-fold cross-validation that uses stratified sampling instead of random sampling; each fold preserves the original class distribution of the dataset.
- Applies this principle to all K folds, ensuring that each fold is a representative sample of the overall population.
- This overcomes the class imbalance issues commonly associated with simple hold-out and standard K-Fold methods.

If original data is:

90% class A
10% class B

Then each fold will also have:

90% class A
10% class B